

Seamless Tutorial: Contextual State Transition Generation Based on Player Internal Knowledge

In-Chang Baek^{ID}, Taehwa Park^{ID}, and Kyung-Joong Kim^{ID}, *Member, IEEE*

Abstract—In the domain of game artificial intelligence, tutorial systems have seen limited advancement despite their critical role in onboarding players. Traditional tutorials often neglect individual learning differences, leading to ineffective instruction. This study proposes a personalized in-game tutorial generation framework that leverages procedural content generation and student modeling. The system integrates a Monte Carlo tree search-based state transition generator and a player modeling module to dynamically adapt tutorial content based on inferred internal knowledge. The approach is validated through large-scale user testing ($N = 88$) in a commercial-style *Match-3* puzzle game environment. Results show that the personalized generator improves learning by up to 44.4% within short sessions. The findings highlight the potential of adaptive tutorials in accelerating skill acquisition and enhancing game experience through seamless, personalized learning contexts.

Index Terms—Game tutorial generation, Monte Carlo Tree Search (MCTS), Procedural Content Generation (PCG), state transition.

I. INTRODUCTION

GAME tutorials, an essential component of modern games, are designed to teach players core game mechanics and enhance engagement with the main content [1], [2], [3], [4]. Several tutorial design patterns have been proposed based on Mayer's principles [5], which emphasize the timing and modality of information delivery. Context-sensitive guidance [6], [7] is one such approach, offering players helpful information based on the current game context. This method enables players to learn game mechanics naturally as they progress through the game,

fostering immersion and a smoother gameplay experience. Most game tutorials are presented early in the gameplay and offer the same content to all players, regardless of individual learning abilities. Because most existing tutorials are handcrafted by human designers, they often fail to accommodate the diverse learning paces and prior knowledge levels of players [8]. However, as learning ability encompasses both the speed of acquisition and existing knowledge, tutorial content should be adapted to match the individual needs of each player.

Several approaches have been proposed to generate tutorial content in an automated manner. Tutorial content has been delivered using various machine learning techniques, such as action recommendation [9], tutorial level generation [10], [11], [12], and contextual instruction [13], [14]. Tutorial level generation is a subfield of procedural content generation (PCG). The tutorial level generation studies have explored this approach in platformer and *Rogue*-like games using evolutionary algorithms. These studies have generated multiple levels designed to teach specific game skills, significantly reducing the manual effort required for tutorial level design. However, a key limitation remains: no prior work has focused on individual learning abilities or generated personalized content that adapts to diverse player skill levels.

To address this problem, we propose a novel tutorial generator—a controllable state transition mechanism—that produces learnable contexts tailored to the player's internal knowledge. We enhance opportunities for context-sensitive guidance by modifying the game state in a seamless manner, ensuring that the player remains unaware of any manipulation of the transition function. The Monte Carlo tree search (MCTS)-based transition function employs controllable content generation techniques to optimize a personalized objective function. The key idea of the proposed method is to incorporate a parameterized reward function [15], [16] into MCTS-based level generators [17], [18], [19] to serve the tutorial purpose.

In addition, we introduce two types of player knowledge models that infer the player's internal understanding based on quantitative metrics and behavioral patterns. By leveraging these models, the generator dynamically produces gameplay contexts that facilitate the learning of unmastered mechanics, reduce redundant repetition, and improve overall skill acquisition efficiency.

The proposed generator was evaluated through both agent-based playtesting and a large-scale user study. We validated our method in a commercial *Match-3* puzzle game setting and tested it in a real-world crowdsourced environment involving

Received 4 April 2025; revised 17 June 2025 and 23 July 2025; accepted 26 July 2025. Date of publication 5 August 2025; date of current version 17 December 2025. This work was supported in part by the Ministry of Science and ICT, Korea, IITP under Grant RS-2025-25410841, in part by the Ministry of Culture, Sports and Tourism in 2025 under Culture, Sports, and Tourism R&D Program through the Korea Creative Content Agency under Grant RS-2024-00441523, in part by the Korea government (MSIT) through the Institute of Information and Communications Technology Planning & Evaluation (IITP) under Grant 2019-0-01842, in part by Artificial Intelligence Graduate School Program (GIST), and in part by the National Research Foundation of Korea (NRF) through Korea Government (MSIT) under Grant RS 2025 16902996. Recommended by Associate Editor D. Loiacono. (Corresponding author: Kyung Joong Kim.)

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Institutional Review Board of Gwangju Institute of Science and Technology under Approval No 20220628 HR 67 18 04.

The authors are with the Department of AI Convergence, Gwangju Institute of Science and Technology, Gwangju 61005, South Korea (e-mail: in-chang.baek@gm.gist.ac.kr; taehwa.p@gm.gist.ac.kr; kjkim@gist.ac.kr).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TG.2025.3596055>, provided by the authors.

Digital Object Identifier 10.1109/TG.2025.3596055

TABLE I
PREVIOUS STUDIES ON GAME TUTORIAL GENERATION AND THEIR APPROACHES

Game	Reference	Algorithm	Online	Output Type	Pers. Factor	Tutoring Method	Evaluation
Gomoku game	[9]	Reinforcement learning	✓	Player action	Play action	Action suggestion	✗
Platform games	[12]	Evolutionary (MAP-Elites)	✗	Level (layout)	✗	Mandatory action	Playtesting
Dungeon games	[11]	Evolutionary (GA)	✗	Level (layout)	✗	Mandatory action	Playtesting
Platform games	[10]	Evolutionary (FI-2Pop)	✗	Level (layout)	✗	Mandatory action	Playtesting
Eight arcade	[13]	Graph representation	✗	Instruction (text)	✗	Instruction	Readability, playtesting
Strategy games	[14]	Natural language processing	✓	Instruction (text)	Player action	Action suggestion	User study ($N=10$)
Tile-matching puzzles	Ours	Search-based (MCTS)	✓	Level (layout)	Player action, skill acquisition	State manipulation, action suggestion	Playtesting, user study ($N=88$)

The table summarizes the game genre, algorithms, online generation capabilities (Online), content output types, personalization factors (Pers. Factor), tutoring methods, and evaluation methods.

participants with diverse ages and backgrounds. Compared to both the original game’s transition function and nonpersonalized transition baselines, our method showed substantial improvements in learning performance. The results demonstrate that the proposed seamless tutorial generation method has the potential to enhance learnability and support real-world deployment.

Our contributions are summarized as follows.

- 1) We propose a tutorial generator that seamlessly adapts game state transitions based on the player’s knowledge.
- 2) We introduce two player models to personalize learning using quantitative and behavioral data.
- 3) We validate our method through simulation and a large-scale user study, showing improved learnability over baselines.

The rest of this article is organized as follows. A literature review on game tutorial generation and existing puzzle level generators is presented in Section II. The definition of game skills is provided in Section III. The personalized tutorial generation method is described in Section IV. The proof-of-concept of the method is validated through an agent-based playtesting approach in Section V. The generator is evaluated through a user study, and the results are presented in Section VI. Section VII provides discussion. Finally, Section VIII concludes this article.

II. BACKGROUND

A. Context-Sensitive Tutorial

Game tutorials aim to teach players specific game systems, such as game rules, skills, and worlds [20]. Game level designers use various learnability design strategies to design immersive tutorial content for a particular game. Ibrahim et al. [21] reviewed several tutorial design methodologies from the perspective of cognitive theory based on *Mayer’s principles of multimedia learning* [5]. The game context is an important step in providing tutorials to players in a natural manner. Context-sensitive guidance [6], [7] gives players helpful hints based on spatial and temporal principles. Tutoring is processed by providing textual information when a player faces a particular learning context, a state in which game mechanisms can be used.

Particularly designed game levels (e.g., worlds and stories) are provided to players to give them the learning context, or they explore the game to reach the learning context. The use of predesigned levels ensures that players obtain the learning context, but repetition can easily cause boredom. Approaches

based on exploring the learning context are less problematic in terms of repetition, but players may not reach the learning context needed to learn a new skill or may not have enough repetition to become proficient in a mechanism. Accordingly, the proposed technique provides the learning context to players in an “invisible” manner to boost the benefits of context-sensitive guidance.

B. Procedural Tutorial Generation

The tutorial generation studies primarily focus on creating universally applicable tutorials for all players. As shown in Table I, previous research on tutorial generation has explored various types of games, algorithms used to generate tutorials, personalization factors considered, and evaluation methods. Existing tutorial generators typically follow one of two approaches: either creating tutorial levels that enforce mandatory actions to teach specific game mechanisms or suggesting actions during regular gameplay without altering the game’s core structure.

One common approach is to recommend actions that efficiently solve given game states, guiding the player by suggesting optimal actions based on the current context. The authors in [9] and [14] proposed methods that follow this approach, aiming to support players by providing context-sensitive hints. Specifically, Zhang and MacLellan [9] directly recommend player actions, offering clear and actionable guidance, while Allameh and Zaman [14] used natural language to deliver flexible and context-aware advice. Instead of generating predesigned learning contexts for specific skills, these methods facilitate natural learning through gameplay by dynamically suggesting efficient actions. This approach helps players develop problem-solving skills as they progress through the game.

Another approach involves generating tutorial levels specifically designed to enforce the use of particular skills. For instance, the authors in [10], [11], and [12] proposed tutorial level generators for platform games that create levels where players must use a specific game mechanism to advance. These levels are deliberately structured so that utilizing the required skill is essential for completion, effectively guiding the player to learn and practice the intended mechanic. By enforcing mandatory actions within the tutorial level, this approach ensures that players acquire the targeted skills through repeated practice within a controlled environment.

Our method combines the strengths of both existing approaches by dynamically adjusting the game state to provide

nonmandatory and flexible learning opportunities. Instead of explicitly enforcing mandatory actions, our approach seamlessly guides players towards states where skill acquisition is possible, while giving action suggestions in a seamless manner. This approach increases the frequency of action suggestions to enhance learning while ensuring that players can freely engage in gameplay, maintaining a balance between guidance and player autonomy. To validate our method, we conducted a large-scale user study involving diverse real-world participants, an effort that has not been attempted in previous studies.

C. Puzzle Level Generators

De Kegel and Haahr [19] reported a meta-study on studies of puzzle content generators. Previous studies employed evolutionary algorithms, such as genetic algorithms (GA) [11], [22] and the multidimensional archive of phenotypic elites (MAP-Elites); search-based algorithms, such as MCTS [23], [24], reinforcement learning (RL)-based algorithms [25], answer-set programming (ASP) [26], and grammar-based approaches [27], [28]. These studies focused on several types of puzzle games, such as spatial reasoning puzzles (e.g., *Sokoban*), mazes, path-building puzzles, and narrative puzzles, to generate game levels in a preliminary phase. The controllability of the generators mainly addressed the difficulty of the levels or a specific action sequence. However, no generator has considered the individual player's knowledge state.

Existing puzzle studies primarily focus on generating puzzle levels in a preliminary phase, meaning that levels are generated offline before the player starts the game. However, online generation capability is essential for producing adaptive in-game content without disrupting gameplay. MCTS, RL, ASP, and grammar-based methods have been applied to puzzle level generators to ensure online generation capability. The RL-based approaches could be suitable for tutorial level generator; however, their generalization to unseen game states should be addressed for real-world evaluation. Among them, MCTS-based methods are particularly advantageous as they do not require domain knowledge on how to manipulate game levels and exhibit fewer generalization issues, relying solely on an objective function to evaluate content fitness. In contrast, ASP and grammar-based methods necessitate explicit domain knowledge to define logical rules or structural patterns, making them less flexible for adaptive content generation. On the other hand, GA methods require substantial computational resources, making them impractical for in-game use.

III. GAME SKILL TUTORING

A. Game Skills in Video Games

Game skill refers to the knowledge, expertise, and abilities that a player acquires and develops to excel in a particular video game. These skills are designed based on the types of actions or timing combinations that players can perform, and usage is learned through repeated learning. Game skills are designed in various patterns for each genre to immerse people in games through physical or logical concepts. For example, in platform

games (e.g., *Super Mario Bros.*), game skills encompass the knowledge required to navigate the game world efficiently. For instance, players can develop the skill of passing through a map with a specific layout and finding shortcuts. This skill may involve recognizing hidden blocks that contain useful items, knowing when to execute precise jumps, or understanding enemy behaviors to clear obstacles strategically. As for fighting games (e.g., the *Street Fighter* series), game skills pertain to the mastery of combat mechanics and tactics. Players need to acquire skills, such as the execution of special moves, combos, and effective blocking. For example, a skilled player may know how to perform a character's complex attacks, which involve combinations of key inputs and timing.

Tutorials are meticulously crafted to impart game skills to players systematically, often through hands-on experiences or demonstrations. As shown in previous studies on tutorial generators [11], [12], level generators aim to suggest to players specific levels where they can acquire hands-on experiences related to a particular game skill. Previous work on tutorial level generation defined several game skills that were usable in their environment and evaluated such levels by ensuring that the agent used a specific skill to clear these levels.

B. Game Skill Definition

We define game skill as a distinguishable execution method for a particular game mechanism. Game skills are learned through pattern recognition of game entities and acquired through repetitive direct or indirect experience. To teach and evaluate game skills, we consider games that have explicitly countable skills and turn-based actions, which eliminate time constraints to accurately assess the player's learned skills.

Game environment: We employ a commercial *Match-3* game *Lollipop & Marshmallow Match3*¹ (LMM3) to evaluate the method in a real-world setting. The game has six patterns of special matching beyond the regular matching. We develop a clone version of LMM3 that enables the content generation interface, containing the game skills in the commercial game. The images of the original LMM3 and the clone game are shown in Fig. 1.

Context-sensitive hint: Both games provide players with context-sensitive hints on the given game board. The hints are presented as animations to guide players toward performing specific actions, teaching game skills to match special blocks. To prioritize the player's actions, the hint is displayed $T_{\text{Hint}} = 5$ s later, ensuring it is given only when the player needs assistance.

State transition: The states of the *Match-3* game board transition according to the following procedure repeatedly [see Fig. 2]: Fig. 2(a) the player swaps two blocks; Fig. 2(b) the matched blocks are destroyed if they consist of three blocks or a special match, and a special block spawns if the player matches a special block pattern; Fig. 2(c) the blocks left suspended by the destroyed blocks drop to fill the spaces vacated by the destroyed blocks, and empty spaces appear in the upper part of the board; Fig. 2(d) new blocks are generated to fill these empty spaces. In

¹[Online]. Available: <https://play.google.com/store/apps/details?id=com.bitmango.go.lollipop2match3>

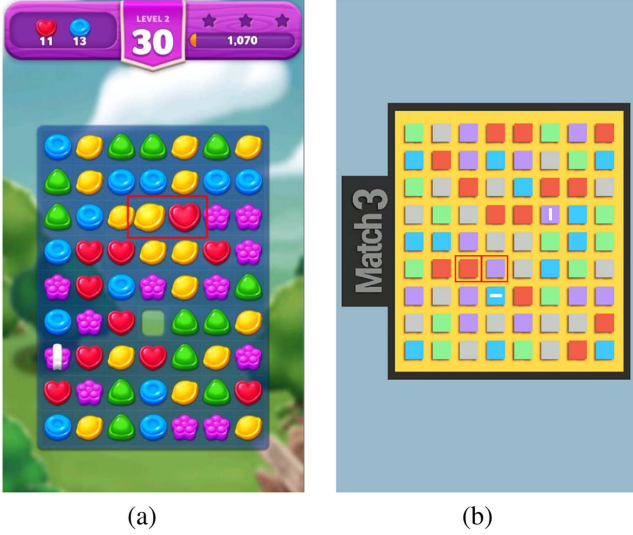


Fig. 1. Screenshots of the (a) original LMM3 and the (b) PCG-enabled clone implementation. The context-sensitive hints emphasize the recommended actions with the highlighting animation. The blocks that have patterns (e.g., horizontal) are considered special blocks. (a) Original game. (b) Clone implementation.

TABLE II
LMM MATCH-3 GAME FEATURES SIX GAME SKILLS

Skill	Usage	Correspondence
S_1	Match four same-color blocks vertically	
S_2	Match four same-color blocks horizontally	
S_3	Match same-color blocks in a T-shaped pattern	
S_4	Match same-color blocks in a 2×2 pattern	
S_5	Match same-color blocks in an L-shaped pattern	
S_6	Match five same-color blocks vertically or horizontally	

The corresponding tile is spawned when the player uses a game skill. The color of the special block corresponds to the color of the matched normal block.

general, the commercial games adopt a random tile generator to ensure game stochasticity.

Game skills: Table II describes six representative game skills in LMM3, particularly their names and usage. For example, the original LMM3 has several designs of a level that explicitly induce matching in advance. The skills are denoted as S_i , where i is the skill number index. Usage denotes the matching patterns of each skill, and a corresponding special block is spawned if a player matches a specific pattern within one of the usages. To provide a hint for matching patterns, the game highlights two tiles to swap [see Fig. 2(a)] after a hint time threshold T_{hint} .

C. Problem Definition

The primary objective of the skill tutoring task is to help players become proficient in various game mechanics within the available gameplay duration. A key challenge in this study is enabling players to acquire six predefined game skills within a restricted time frame. To address this, we propose a tutoring approach that seamlessly integrates into the gameplay, ensuring that players encounter learnable states without requiring additional user interface elements or explicit guidance. We investigate how modifying game state transitions influences players' ability to grasp game mechanics efficiently. Since transition

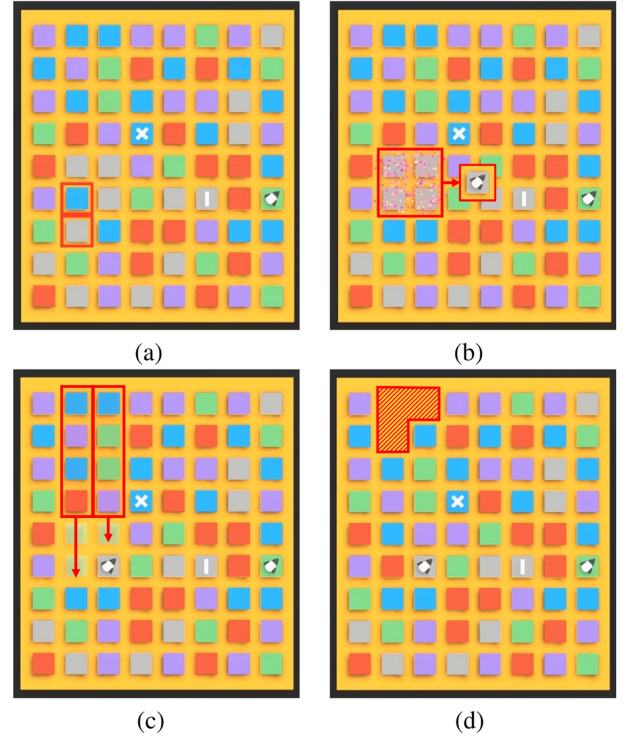


Fig. 2. Game board transition in *Match-3* game environment. (a) Player swaps two blocks. (b) Specific pattern spawns a special block (e.g., rocket). (c) Upper blocks drop down to fill the empty spaces. (d) New blocks spawn at the top to fill the empty spaces. The generator manipulates the game transition by determining the color of new blocks. (a) Swap blocks. (b) Match special blocks. (c) Drop blocks. (d) Generate new blocks.

functions are a fundamental aspect of most games, we suggest that our state adaptation technique could be applied across different game genres.

IV. PERSONALIZED TUTORIAL GENERATION

This study introduces a machine learning-based approach to dynamically adjusting game states, assisting players in skill acquisition while preserving the original gameplay experience. The architecture comprises two main components: 1) the player student model (PSM), which represents the internal state of the player's knowledge and 2) the context generator, a machine learning-based state transition function. The context generator maximizes the probability of learning based on the individual's learning progress. Since no additional user interface elements or tutorial scenes are required, this tutoring method ensures that players perceive the experience as close to the original gameplay, making the adaptation process unobtrusive.

Fig. 3 illustrates the overall process of the proposed architecture. The input of the procedural context generator consists of two elements: the PSM and the game state. The generator outputs the next game state with the highest probability of facilitating learning. In this process, the next state is determined dynamically based on the player's current knowledge state.

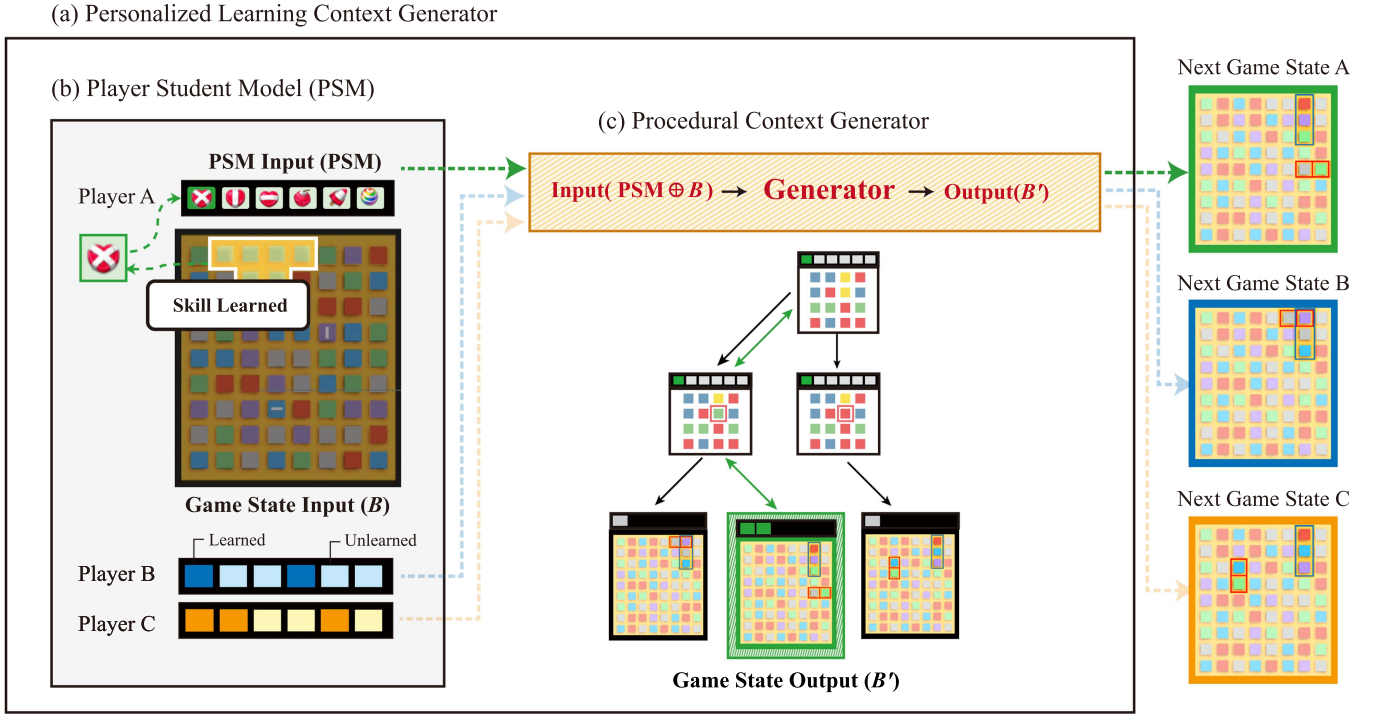


Fig. 3. Overall architecture of personalized tutorial generator. (a) Personalized learning context generator incorporates. (b) PSM. (c) Procedural context generator. The next game states are generated based on the status of the individual PSM.

A. Level Representation

The game state is represented as a 3-D matrix, which then serves as the input of the context generator. The game board state, B , is denoted as $B = [H, W, N_{\text{property}}]$, where W is the matrix width, H is the matrix height, and N_{property} is the number of properties. N_{property} consists of integer values that determine the characteristic of a game block. In this work, we set $N_{\text{property}} = 2$ because the blocks in LMM3 have two characteristics (color and type). In B , the player's behavior transitions through regular game logic; in the next board state (B'), the player is asked for action. The context generator takes B as its input and returns B' , a portion of which is additionally changed, to provide the learning context. The nonempty tiles in B are frozen so that the generator cannot overwrite them.

B. Player Student Model

We propose the PSM, which tracks the game learning progress to generate adaptive learning contexts for individual players. A student model is a core component of the intelligent tutoring system (ITS) framework [29]. An ITS is a computer-based training tool for educating humans on specific skills used in various fields, such as image editing tool [30], software design [31], and sketching [32]. Here, the PSM captures and updates information about a learner's training progress, enabling the system to provide personalized, adaptive instruction.

The concept of the model is used to track the player's learning by computationally representing their progress in each game skill. The player's skill learning progress is denoted in binary form as $\text{PSM}(S_i)$, where S_i is the i th skill element in the skill

TABLE III
COMPARISON OF TWO TYPES OF STUDENT MODELS BASED ON THE OPERATION LOGIC AND REQUIREMENT

	Operation Logic	Requirement
Threshold	Quantitative assessment via playtesting agents	pre-collected threshold values
Self-efficacy	Behavioral data analysis from real human	-

set S ; the number 1 indicates the learned skill, and 0 is the opposite. We incorporate the $\text{PSM}(S_i)$ term into the generator's objective function for the generation of learnable game states. We propose two types of PSM modules that track the learning progress implicitly, namely, threshold-based and self-efficacy PSMs. The features of the two student model types are described on Table III.

1) *Threshold-Based PSM*: The threshold-based PSM [PSM_{TH} ; (1)] determines the player state using the player's required repetition count to learn a particular skill. The current number of repetitions for skill i is $\text{Matched}(S_i)$, and the required number of repetitions to complete the learning of skill S_i is represented by a threshold, $\text{Threshold}(S_i)$. If the number of skill-use repetitions exceeds the threshold number, then PSM_{TH} returns the learning status of skill S_i as "learned." This type of learning progress verification method can be used when the amount of training can be determined in advance

$$\text{PSM}_{\text{TH}}(S_i) = \begin{cases} 1, & \text{if } \text{Matched}(S_i) \geq \text{Threshold}(S_i) \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

2) *Self-Efficacy PSM*: The self-efficacy PSM [PSM_{SE} ; (2)] determines the player state based on the human player's behavioral patterns. PSM_{SE} is designed following self-efficacy theory [33], which refers to an individual's belief in their ability. After a player has learned a specific skill, they tend to make decisions independently rather than relying on given hints. This type of learning progress verification method is useful in user studies where threshold values cannot be predetermined.

Initially, PSM_{SE} is set to the default value "unlearned" for all game skills when a player starts the game. To update the model, we collect game logs containing decision time, actions performed, and hint actions. $PSM_{SE}(S_i)$ is updated ($\leftarrow$) to "learned" if the player executes S_i using their own ability, either by performing the skill before the hint appears or by executing a different action from the hint after it appears.

The student model updates its state when one of the following conditions is satisfied.

- 1) The player successfully performs the game skill S_i within the decision time before the hint appears ($T_{decision} < T_{hint} \wedge (A_{player} = S_i)$).
- 2) The player performs an action that differs from the hint action, demonstrating independent decision-making ($A_{hint} \neq A_{player} \wedge (A_{player} = S_i)$).

Therefore, the update condition can be summarized as follows:

$$PSM_{SE}(S_i) \leftarrow \begin{cases} 1, & \text{if } [(T_{decision} < T_{hint}) \wedge (A_{player} = S_i)] \\ 1, & \text{if } [(A_{hint} \neq A_{player}) \wedge (A_{player} = S_i)] \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

In summary, PSM_{SE} determines whether a player has learned a skill based on their ability to perform it independently, either by acting before the hint or by choosing a different action despite the hint. This adaptive approach is beneficial in scenarios where the skill acquisition process varies between individuals, and fixed thresholds are not applicable.

C. Puzzle Generator Models

1) *Context Generation*: The context generator is represented as a dynamic transition function, f . This transition function is invoked after the player's action and the essential game logic. In *Match-3* games, instead of using a random function, the context generator determines how empty tile locations are updated (see Fig. 2). The context generator takes the board state B , updates the state based on the player's action and the PSM state, and returns a revised state: $B' \leftarrow f(B|B, A_{player}, PSM)$. The transition function is adapted based on the player's internal knowledge state.

2) *MCTS-Based Context Generator*: MCTS [34] is a simulation-based tree search algorithm widely used in sequential decision-making problems. We formulate the generation of the next game state through the sequential transformation of the game state into an MCTS exploration procedure. MCTS uses a utility function to evaluate a node v , upper confidence bounds applied to trees [UCT; (3)], to visit valuable nodes frequently while decreasing the exploration ratio. MCTS selects the next

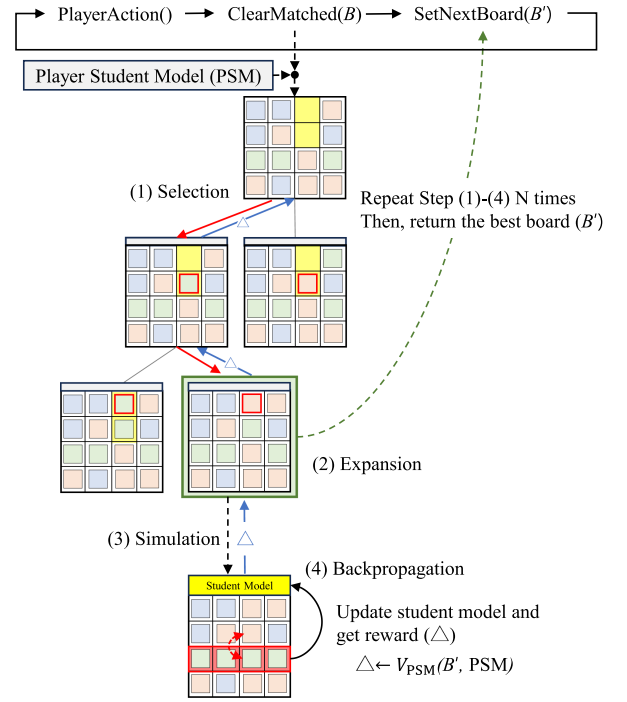


Fig. 4. Overview of the MTCS process guided by the student model, where the player's knowledge state influences the selection, simulation, and backpropagation to generate a personalized next game state.

child node using the UCT value during exploration, N_p and N_v are the visit counts of the parent and current nodes, respectively; C is the exploration ratio factor; and $\hat{X}(v)$ is the accumulated node value. The algorithm updates the tree by repeating the following sequence over N_{Sim} iteration

$$UCT(v) = \frac{\bar{X}(v)}{N_v} + C \sqrt{\frac{\ln(N_p)}{N_v}}. \quad (3)$$

Fig. 4 illustrates the search process of the MCTS-based generator model. There are two empty tiles in B . The MCTS-based context generator model is updated by repeating the following steps.

- 1) *Selection*: A terminal node is selected by searching through the tree to identify a state that is beneficial for learning. A child node is then chosen using UCT, where the state is evaluated iteratively.
- 2) *Expansion*: A new child node is created from the selected terminal node. This process mutates part of the parent state by filling a vacant location with a randomly sampled tile type.
- 3) *Simulation*: If the terminal node is a playable game state, an agent simulates the given game state, and the reward is measured using the board state value function, V .
- 4) *Backpropagation*: The method updates the value of the traversed node with the reward obtained from V . The updated node values influence the selection order of nodes in the next iteration.

In general, a game state B has an average of 4.11 (± 2.38) empty spaces. The complexity of the search space varies with the number of empty spaces. The mean complexity is approximately $6^{4.11} \approx 1.578 \times 10^3$, where 6 is the number of block colors. The

upper bound is $6^{11.25} \approx 5.678 \times 10^8$, where 11.25 is the maximum value of 99.5% of the distribution. An efficient method for complex search spaces reduces the time spent on board value measurement, allowing the context generator to return a learning context in a limited execution time.

D. PSM-Driven Puzzle Generative Models

1) *Board State Value*: The board state value function, V , measures the value of B to estimate the value of a learnable state. A learnable case refers to the conditions under which players can use their game skills by performing particular actions. V calculates the number of cases in which a player can learn a skill in a game state. The basic board state value function, V_{GEN} [see (4)], calculates the number of learning cases for all skills, B is the game state, S is the skill set, and S_i is the i th skill element

$$V_{\text{GEN}}(B) = \sum_{S_i \in S} (\text{Learnable}(B, S_i)). \quad (4)$$

The utility function, $\text{Learnable}(B, S_i)$, returns a binary value according to whether the player utilizes skill S_i in B . This function can be transformed into a useful condition for a player to use a particular skill, or be used to determine whether the availability of that skill is activated by a particular condition. It returns 1 if the game state has a matchable condition—the chance of learning skill S_i —for a specific skill in the current game state and is 0 otherwise.

To check whether a particular skill is available in a given game state, we simulate all available actions in the game state. Because simulating this process takes time, the state should be efficiently generated and evaluated in procedural generation. The utility function V is utilized by the context generator as an objective function. The generator leads the player to a more useful learning context, which has a particular condition for learning a specific skill, than a typical game transition.

2) *PSM-Based Board State Value*: The personalized board value function V_{PSM} is expressed in (5). We apply a masking term, $(1 - \text{PSM}(S_i))$ to adapt the learning context to the unlearned skills. This, V_{PSM} evaluates the game state within the context of valuable learning opportunities, allowing players to leverage their unlearned skills. The board state value is personalized by the PSM model, which automatically tracks the learning progress of each player

$$V_{\text{PSM}}(B, \text{PSM}) = \sum_{S_i \in S} (\text{Learnable}(B, S_i)) \cdot (1 - \text{PSM}(S_i)). \quad (5)$$

V. PROOF-OF-CONCEPT WITH HUMAN-PROXY AGENTS

The primary goal of this section is to assess generators' performance by simulating the learning of the players via playtesting methodology. The secondary goal is to figure out the appropriate generator model for real-time generation. The generator model must return the learnable context within a restricted time frame to maintain interactivity in the game. Since it must transition to the next game state before the player experiences interruptions, balancing the time required for generating appropriate learning content and computational

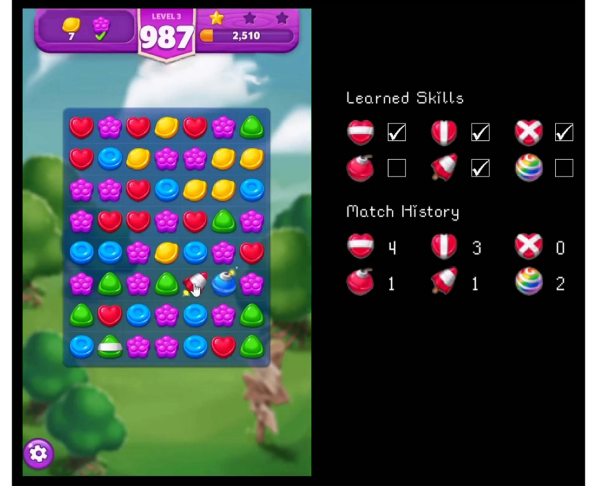


Fig. 5. Game client to collect threshold values from human players. The players play the LMM3 game and check if they find the match patterns from the repetitive learning.

efficiency is crucial. We evaluate the models based on their capability to improve the player's learning speed and their generation time to identify the most efficient model.

A. Human-Proxy Agents

The players have different abilities to learn game skills and require different numbers of task repetitions to learn a skill. The threshold-based PSM, PSM_{TH} (see Section IV-B1), estimates the learning progress using the repetition count for a game skill and the minimum value needed to complete the learning.

Threshold parameter values are acquired from human game-play to construct a play-testing agent that represents the human players' PSMs. We recruited 11 participants from a local institution and ask them to play LMM3 (see Fig. 5) with a self-reporting task. The study was approved by the Institutional Review Board. The participants are asked to play the game for 10 min and immediately report to the experimenter when they acquire the game skill S_i and the game records the repetition count to use the game skills.

The collected threshold values show variation among players, reflecting differences in learning abilities. On average, players required 7.91 ± 2.55 repetitions for S_1 , 6.18 ± 2.64 for S_2 , 2.00 ± 1.95 for S_3 , 2.36 ± 1.03 for S_4 , 7.45 ± 5.05 for S_5 , and 3.18 ± 1.08 for S_6 ; the actual parameter values are described on Supplementary Material. These results indicate that the average number of repetitions and their variability differ based on the intuitiveness and rarity of each skill. Using these individual threshold parameters, we construct 11 distinct PSM_{TH} models that mimic the diverse learning abilities of human players. These models serve as proxies for human players, enabling us to simulate a range of learning behaviors within the agent-based system.

B. Comparisons

Random: A widely used transition algorithm in *Match-3* games that fills empty spaces with randomly colored normal blocks, without considering learning optimization.

TABLE IV
COMPREHENSIVE RESULT OF THE HUMAN-PROXY EXPERIMENT

Model	Board Value	Action Count ($\downarrow$)	Time (s, $\downarrow$)
Random	-	166.43 (± 43.81)	0.00 (± 0.00)
Sampling-PSM	0.11 (± 0.29)	136.67 (± 50.05)	0.45 (± 0.04)
MCTS-GEN	0.49 (± 0.32)	142.96 (± 51.89)	0.16 (± 0.18)
MCTS-PSM	0.11 (± 0.57)	122.25 (± 43.81)	0.15 (± 0.16)

Sampling-PSM: Similar to *Random*, but evaluates board states using the personalized value function V_{PSM} . This generator uniformly samples tile types to generate N_{sample} playable levels and selects one as the next state.

MCTS-GEN: An MCTS-based context generator that uses V_{GEN} as the board value function. This method does not consider personalization and assumes all skills must be learned equally.

MCTS-PSM: An MCTS-based context generator that incorporates a personalized board value function, V_{PSM} . This model transitions the game state to prioritize opportunities for learning unmastered skills.

The PSM_{TH} method is employed to simulate the player's knowledge model within the agent-based playtesting procedure.

C. Experiment Setup

The initial game state is set to normal blocks of random colors to mitigate performance measurements that are biased against a particular game state. A total of 4.4 million frames (11 PSMs $\times$ 500 episodes $\times$ 200 episode length $\times$ 4 generators) are simulated with the agents. The game log data include the board state value, the step count of the skill learned, and the generation (execution) time. The parameter of Sampling-PSM is set to $N_{\text{sample}} = 5$ to ensure a comparable timeframe with MCTS-based methods; the number of simulations for the MCTS models is set to $N_{\text{sim}} = 200$.

D. Experimental Result

The learning speed measures how fast the PSM-based generator can teach a player. We record the action count (AC) when the matched game skill ($\text{Matched}(S_i)$) reaches the threshold value ($\text{Threshold}(S_i)$); the AC is recorded when all skills (S_{1-6}) are marked as "learned." The descriptive results are summarized in Table IV.

AC is the recorded AC needed by the player to reach the threshold (i.e., to complete skill learning). A low value means that the player learns all skills in a short time. The normalized improvement of a generator (G) is measured through the relative decrease rate in the AC of random [see (6)]. MCTS-GEN decreases the AC by 14.10% (23.47 steps), whereas MCTS-PSM decreases it by 26.55% (44.18 steps). The comparison of MCTS-GEN and -PSM shows that the models with PSMs teach the players through fewer actions

$$\text{Improvement}(G) = \left(\frac{\text{AC}_{\text{Random}} - \text{AC}_G}{\text{AC}_{\text{Random}}} \right) * 100. \quad (6)$$

Board value denotes the mean board value computed by the generator. The MCTS-GEN shows higher values because the

characteristic of the general board value function (V_{GEN}), which has no PSM, assumes all game skills are valuable [see (4)], however, requires higher ACs despite higher board value. Thus, although these generators generate as many skills as possible, they are not helpful enough to accelerate learning.

Time denotes the mean execution time for each generation step. The response time should be lower than 1 s to ensure that users feel the system is reacting immediately, maintaining their flow of thought and preventing frustration [35]. A significant computation time for state transitions can lead to game malfunctions, making it crucial to return the next game state within a limited time. The MCTS-based level generators perform within 20 ms, suggesting that applying the context generation-based tutorial methodology does not cause significant frame drops.

VI. USER STUDY EXPERIMENT

A. Comparisons

The comparison comprises three generators—*Random*, *MCTS-GEN*, and *MCTS-PSM*—to investigate whether the personalized generator improves the learning ability in the real-world setting. However, it is impractical to use threshold-based PSM in the user study due to it is not easy to acquire the threshold value in preliminary from the unseen participants. Accordingly, we selected the self-efficacy PSM model, PSM_{SE}, to infer the player's internal knowledge in a seamless manner.

MCTS-PSM: A variant of MCTS-based generation that incorporates personalization by optimizing the board state value based on PSM_{SE}. This model determines the player's knowledge based on the threshold and repetition count. We set the decision time to $T_{\text{hint}} = 5.0$ s and $T_{\text{decision}} = 5.5$ s to determine whether the player learned a specific skill. The additional 0.5 s in decision time accounts for the perceptual and cognitive processing required to interpret the hint.

We provide context-sensitive guidance to users which are widely used on the commercial *Match-3* games. Each hint, which appeared after the player failed to make any decision within T_{decision} , highlighted two blocks to do a swap [see Fig. 2(b)]. This guidance was commonly provided regardless of the generator type, and the greedy heuristic agent made the action recommendation. The transformed state, combined with context-sensitive hints, naturally guided the player to explore alternative actions and facilitated the discovery of new game mechanics. We also incorporated this guidance system into our experiment software because it is a basic tutorial strategy in *Match-3* games.

B. Participants

We invited 100 participants from Amazon Mechanical Turk, a crowdsourcing platform, and 88 agreed to join. The average age of the participants was 32.9 years (SD = 6.00), and the sample included 48 males and 40 females. We paid them \$5.00 for 30 min of participating and gave a bonus to the top 10% of the players to promote their willingness to learn.

The distributions of individuals' ages and experiences with *Match-3* games are shown in Fig. 6. The majority of participants (89%) were classified as hobbyist gamers, engaging in gameplay

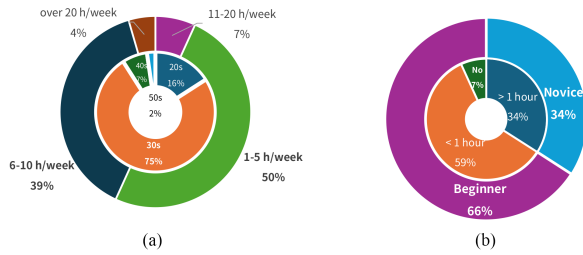


Fig. 6. (a) Subjects' ages and general expertise on games. (b) Prior experience on *Match-3* games and experiment groups. (a) Age, Game time (weekly). (b) *Match-3* Experience, Group.

for fewer than 10 h per week. We also assessed the participants' experience with *Match-3* games. In total, 59% of the participants had less than one hour of experience, and 7% had no experience at all. The remaining 34% reported having spent more than one hour playing *Match-3* games. To facilitate further analysis, we categorized participants into two groups based on their prior *Expertise*: those with one hour or less of experience were labeled as *Beginner* (66%), while those with more than one hour were labeled as *Novice* (34%). Although 93% of the participants had previously played *Match-3* games, differences in usage patterns, visual styles, and in-game item effects necessitated additional learning for players to acquire the game skills in the experimental game.

The user study was conducted between-subjects due to the learning effect; a participant was randomly allocated one of the three generative methods. Random was the control group, and MCTS-GEN and MCTS-PSM were the experimental groups. A total of 33 participants underwent game tutoring using Random, 28 underwent MCTS-GEN, and 27 used MCTS-PSM. The participants were unaware of the game tutoring method assigned to them.

C. Experiment Setup

Procedure: The user study was conducted in the following sequence: 1) administering a pregame survey (to obtain the participants' basic information and gaming experience), 2) learning *Match-3* game skills using a generator model, 3) solving 12 quiz levels to evaluate learning, and 4) conducting an after-game survey to ask about their game experience.

Evaluation: We prepared 12 quiz levels (six game skills $\times$ two levels) to evaluate the player's learning in an explicit manner. Given the randomness of the game, testing the players via demonstration-based evaluation was fair. Each quiz level asked the player to match a particular game skill by giving a common game board across all players (see Fig. 7). We analyzed the correct-answer rates for the six game skills.

Log data collection: While the players participated in the user study, we collected 5280 (60 steps $\times$ 88 players) game log data, such as the game board state, player action, use of hints, PSM status (see Section IV-B2), decision time, and execution time of context generation. We published the experiment software online using Unity WebGL,² and the game log data were automatically collected in a database. Finally, we gathered 1056

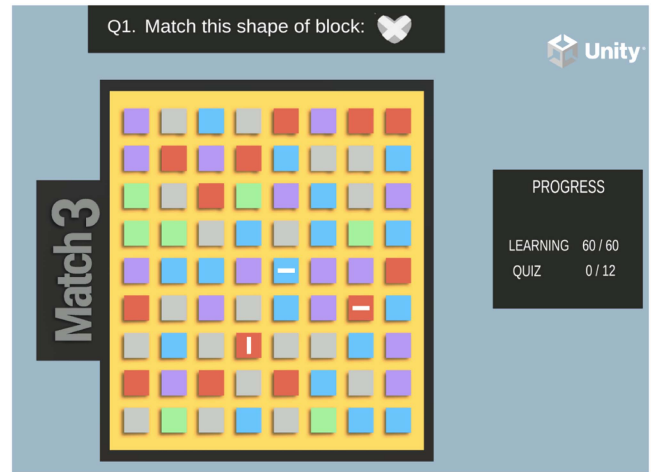


Fig. 7. Quiz scene displayed after the training session, showing the target game skill the player is expected to perform.

game log rows (12 actions $\times$ 88 players) responses from 12 quiz levels. The participants learned the game skills with 60 action chances, and they took approximately 12 min to finish.

D. User Study Result

1) Learning Effect: Learning effect refers to how well players acquire game skills within a limited number of actions. To measure this, we assess players' responses to quiz levels and evaluate whether they successfully perform the targeted game skills. Fig. 8 illustrates the proportion of participants who mastered each of the six game skills. The results are presented for three groups—beginner and novice—as well as across three level generators. A skill is considered mastered if the participant submits at least one correct answer among the two quiz levels. Fig. 9 shows the average number of correct quiz answers for each generator, along with the improvement ratio of MCTS-PSM compared to the Random and MCTS-GEN generators and the independent t-test [36] was used for two-pair comparison statistical analysis.

Fig. 9 shows a noticeable improvement in learning outcomes for the Novice group, who already possess a basic understanding of the game's control mechanics. This suggests that the proposed method is particularly effective for players who possess prior knowledge of the basic controls in *Match-3* games. MCTS-PSM improved the learning effect by 44.4% compared to the Random generator ($t(17.7) = 1.318$; $p = 0.204$), and by 65.1% compared to the MCTS-GEN generator ($t(13.0) = 1.897$; $p = 0.080$). In contrast, MCTS-GEN showed no improvement, with a 12.5% decrease relative to the Random baseline ($t(18.9) = -0.464$; $p = 0.648$). Although no significant difference was observed, a descriptive trend was apparent, suggesting a potential effect that may become more pronounced with stricter control of participants' prior gaming experience and learning ability. These results indicate that MCTS-PSM is more effective for specific groups with relevant prior knowledge, rather than for all players. Furthermore, as seen with MCTS-GEN, nonpersonalized content appears to be less beneficial for learning—even when players are given equal opportunities—if their individual

²[Online]. Available: <https://docs.unity3d.com/kr/2023.2/Manual/webgl-gettingstarted.html>

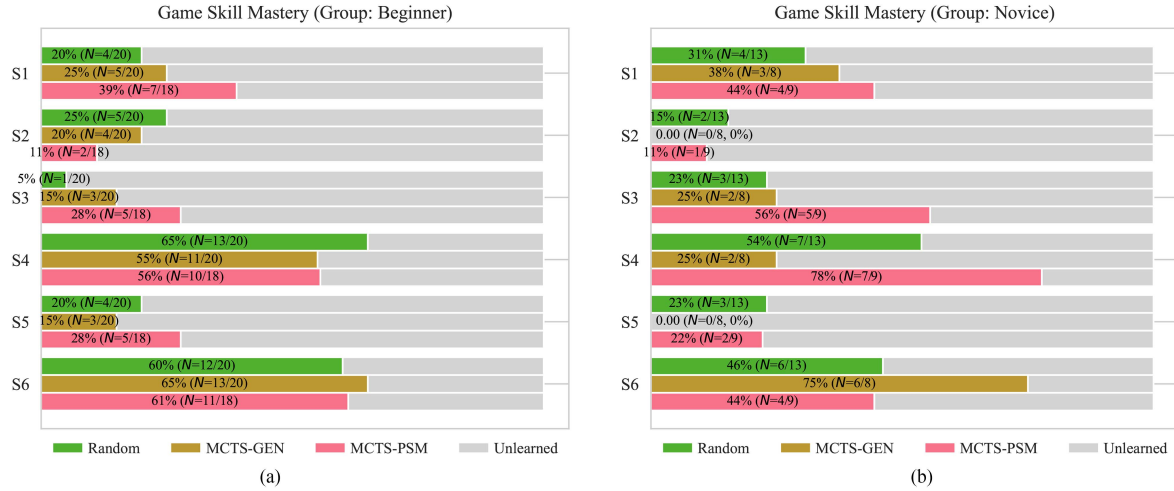


Fig. 8. Skill mastery across the three different expertise groups and three level generators. A skill is considered mastered if the player applied the correct skill at least once across two quiz levels. (a) Beginner group. (b) Novice group.

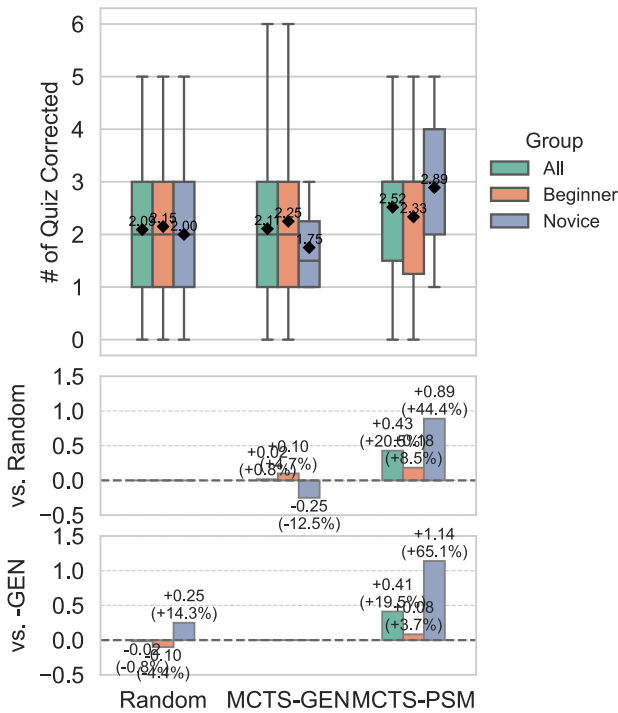


Fig. 9. Number of correct quiz levels across the three different expertise group and three level generators. The two bar graphs below illustrate the relative improvement of MCTS-PSM compared to the other level generators.

knowledge states are not considered. Accordingly, we argue that incorporating a student model is helpful for providing personalized learning contexts and improving learning outcomes. The full results on correct answer rates and statistical tests are summarized in the Supplementary Materials.

2) *Game Experience*: To evaluate the subjective game experience, we conducted a one-way ANOVA [37] on questionnaire responses from novice participants—those who showed the most improvement in learning outcomes. The internal consistency of the questionnaire was good, as indicated by a Cronbach's

alpha of 0.83 [38], suggesting that the items reliably measured a cohesive construct of game experience.

As summarized in Table V, there was no significant difference on game experience as shown on Q1 and Q2. Importantly, despite the enhanced learning support, participants did not report a disruption in gameplay consistency. Specifically, Q5, which asked whether the respawn behavior of blocks differed from that of typical *Match-3* games, showed no significant difference between groups, suggesting that the generator's transitions were not perceived as unnatural. These results indicate that the proposed generator successfully preserves the game experience with the traditional game transition method, while operating in real time, thus making it suitable for integration into commercial game environments.

In addition to preserving the gameplay experience, the MCTS-PSM method also improved perceived learning outcomes. Significant differences are found in Q4 ($p = 0.035$), which assessed the learning speed enhancement of the tutorial session. Post-hoc analysis using Tukey HSD [39] tests revealed that MCTS-PSM led to significantly higher ratings than MCTS-GEN. Although Q3 did not show statistically significant pairwise differences ($p = 0.063$), MCTS-PSM received the highest mean score, indicating a clear positive trend. These results support our claim that the personalized transition function can enhance learning without disrupting the player's immersion or familiarity with *Match-3* game mechanics.

VII. DISCUSSION

A. Comparing Proxy-Human and Real-Human Experiments

Learning effect: In the human-proxy evaluation (see Section V-D), both MCTS-GEN and MCTS-PSM improved learning speed, by 14.10% and 26.55%, respectively. However, in the real-human setting, only MCTS-PSM demonstrated a learning improvement, increasing the correct-answer rate by 44.4%, while MCTS-GEN showed a decline of 12.5%. This discrepancy implies that, while MCTS-GEN facilitates repeated

TABLE V
NOVICE PARTICIPANT RESPONSES TO QUESTIONNAIRE ITEMS (7-POINT LIKERT SCALE)

		Random	MCTS-GEN	MCTS-PSM	F	p
Q1.	I felt immersed while playing the game.	5.92±1.38	5.38±1.69	6.11±1.05	0.648	0.531
Q2.	I learned the basic game mechanics.	4.92±1.75	4.50±1.60	5.56±1.13	1.006	0.379
Q3.	The tutorial session was helpful for learning the matching skills.	5.92±1.26	4.50±2.20	6.00±0.50	3.059	0.063
Q4.	I was able to learn the existing game skills in a short amount of time.	5.69±1.60	4.38±1.51	6.11±0.60	3.798	0.035*
Q5.	I could not distinguish the re-spawned blocks from those in regular match-3 games.	4.77±1.69	5.00±1.41	5.56±1.01	0.797	0.461

F - and p -values are from one-way ANOVA. * indicates $p < 0.05$.

exposure to learning of specific game skills, the lack of personalization may cause players to develop skewed learning focused on a narrow subset of game skills. In particular, it may have caused users to overtrain on a limited subset of game skills or overly focus on specific tile patterns. This result highlights the importance of personalized content generation, as provided by MCTS-PSM, to support more balanced and effective learning in real player scenarios.

Online generation capability: We emphasize the online generation capability of the proposed method for adaptive context generation, as supported by the results of the user study. In the controlled in-lab setting, the MCTS-based generator (MCTS-PSM, $N_{\text{sim}} = 200$) achieved an average generation time of 0.14 s (± 0.15) on a workstation. In the real-world setting, MCTS-PSM and MCTS-GEN recorded average generation times of 0.62 s (± 0.51) and 0.69 s (± 0.58), respectively; outliers beyond two standard deviations ($z > 2$) were excluded from this analysis. Although the generation time increased slightly depending on the hardware specifications of users' devices, both generators maintained a reasonably affordable generation time—under one second—demonstrating their suitability for real-time use in practical use.

B. Limitation on Internal Knowledge Prediction

The ability to predict internal knowledge is fundamental to providing personalized learning experiences; however, it represents one of the key challenges addressed in this study. To estimate internal knowledge without access to precollected data, we adopt a heuristic algorithm grounded in self-efficacy theory. However, this approach is constrained by its reliance on domain-specific knowledge, and its parameters are empirically determined. This section investigates the extent to which the accuracy of internal knowledge prediction influences game skill acquisition. Prediction accuracy is evaluated by comparing the final or paired outputs of the self-efficacy-based algorithm with the correct-answer ratio in the novice player group [see Fig. 8(b)]. We conduct a Pearson correlation analysis [40] to examine the relationship between the learning improvements achieved by MCTS-PSM and the corresponding prediction accuracy.

We observed a moderate positive correlation between prediction accuracy and learning improvement ($r = 0.41$), as shown in Table VI. This suggests that more accurate estimation of internal knowledge can contribute to enhanced personalization in game-based tutoring. Interestingly, recall shows a moderate negative correlation with learning improvement ($r = -0.36$).

TABLE VI
RELATIONSHIP BETWEEN LEARNING EFFECT AND THE ACCURACY OF THE SELF-EFFICACY PREDICTION ALGORITHM

	Imp.	Accuracy	Precision	Recall	F1
S_1	44.44	0.30	0.28	0.81	0.42
S_2	-27.78	0.20	0.17	1.00	0.29
S_3	140.74	0.72	0.36	0.42	0.39
S_4	44.44	0.86	0.84	0.94	0.89
S_5	-3.70	0.53	0.17	0.35	0.23
S_6	-3.70	0.75	0.92	0.63	0.75
Corr. ($r =$)		0.41	0.04	-0.36	0.07

The improvement (Imp.) measures the learning effect enhancement between Random and MCTS-PSM in the Novice group. The Pearson correlation coefficient (r) measures the relationship between the improvement and each metric.

This may suggest that a more conservative estimation—where the system assumes the learner has not yet mastered certain knowledge—can lead to more effective support by ensuring sufficient practice opportunities. These findings highlight the importance of accuracy in internal knowledge prediction, while also cautioning against relying solely on optimistic recall-based assessments. To address overestimation, future work may incorporate verification mechanisms for learned skills and account for the potential forgetting of knowledge over time.

C. Extendibility to Complex Game Skills

This study demonstrated the proposed method within a simple *Match-3* game mechanic that primarily requires pattern recognition skills. While this simplified setting allowed for clarity in evaluating learning predictability and user behavior, it can be extended to more advanced skills, such as planning and reasoning, about the chain effects triggered by tile interactions. If complex game skills can be quantitatively defined and the predictability of learning progress is ensured, adjusting the depth of the search process can enable the system to support the acquisition of more advanced player skills.

We selected a turn-based setting due to its clearly defined cycles of state manipulation and player evaluation, though the method remains applicable to real-time games. For example, in fighting games, skills can be defined through key combinations and timing (e.g., kick and tackle), and more advanced skills may involve predicting the opponent's actions and planning counterstrategies. Further research is needed to identify appropriate intervention timings and suggestion strategies for applying our method in real-time games.

D. Integrating Instruction and PCG Tutorials

Delivering tutorials with the appropriate difficulty and format is essential for maintaining immersion and sustaining player engagement. Traditional scripted tutorials are widely used in games due to their structured design, strong developer control, and high predictability in guiding player learning. Since re-engaging with scripted tutorials typically depends on the player's own initiative, transitioning back into a learning state often requires an understanding of the game mechanics and sufficient control proficiency. To complement this, the proposed PCG-based tutorial generation method provides personalized guidance throughout gameplay. By allowing players to recall and repractice forgotten mechanics interactively, especially after periods of inactivity, it supports retention and reinforces learning in a more seamless manner. While this personalization enhances adaptability, it inherently reduces the developer's control over the tutorial sequence. Our experimental results (see Section VI-D) demonstrate that this approach is particularly effective in enhancing learning outcomes for players who already understand basic game controls.

To leverage the strengths of both approaches, we propose integrating the PCG-based tutorial method with initial scripted tutorial levels, allowing for improved controllability during early learning while maintaining adaptability as the player progresses. Furthermore, incorporating explicit tutorial descriptions into PCG-generated content may improve the predictability of instructional outcomes. This integrated framework offers a complementary balance between control and adaptability, supporting long-term and individualized player learning.

VIII. CONCLUSION

We presented a novel method for personalized tutorial generation that operates through seamless state transition control, enabling real-time adaptation without interrupting the gameplay experience. By integrating player modeling and PCG, our system generates tutorial content dynamically in response to inferred internal knowledge. Through large-scale user evaluation and simulation-based testing, we confirmed that personalized transitions not only improved learning efficiency but also maintained players' sense of immersion and familiarity with the game. Notably, the proposed method was effective in supporting efficient learning for novice players who were already familiar with basic game controls but needed to acquire new game mechanics. By focusing on conceptual rather than operational guidance, the method enhances the player's ability to grasp unfamiliar systems, thereby increasing its transferability to other games with similar interaction schemes but different underlying mechanics.

While the system demonstrates practical effectiveness, it relies on heuristic knowledge models that may limit generalizability. The self-efficacy-based player model, for instance, is sensitive to parameter tuning and behavioral interpretation. Moreover, the method is currently optimized for turn-based puzzle games with discrete skill representations. Future work will focus on enhancing the robustness of internal knowledge inference through learning-based approaches and expanding applicability to broader genres, including real-time and open-world games. In

TABLE VII
SIMULATION RESULTS OF LEVEL GENERATORS

Model	Board Value	Action Count ($\downarrow$)	Time (s, $\downarrow$)
Random	-	166.43 (± 43.81)	0.00 (± 0.00)
MCTS-PSM	0.11 (± 0.57)	122.25 (± 43.81)	0.15 (± 0.16)
GA-GEN	0.77 (± 0.46)	138.81 (± 53.48)	5.88 (± 2.68)
GA-PSM	0.11 (± 0.32)	81.03 (± 40.43)	4.76 (± 2.04)

addition, we aim to explore long-term learning adaptation and its integration with difficulty scaling and narrative progression for richer personalized gameplay experiences.

APPENDIX A GA-BASED LEVEL GENERATOR

To demonstrate the generalization capability of the PSM model, we tested it with different baseline generation algorithms. The GA algorithm evolves an N_{pop} -sized population that consists of the complete game state, B' . The chromosome length is set adaptively according to the number of empty spaces in B . The evolutionary procedure includes roulette selection, single-point crossover, resetting mutation, and elitism. The population evolves by repeating the following sequence over N_{gen} iterations and then returns the best chromosome in the population. The mean and standard deviations of the generation times and ACs are summarized for three baselines—Random, MCTS, GA—in Table VII.

The generation time determines whether an algorithm can be used in real-time applications. The GA-based generator requires 4.76–5.88 s per execution, which is significantly longer than the MCTS-based generator. As a result, we exclude the GA baseline from the user study due to its high execution time, which negatively impacts the user experience. This increased execution time is primarily caused by the need to check playability and evaluate fitness across mutated chromosomes, even though the fitness of unchanged chromosomes is cached. However, GA-PSM significantly improves AC performance compared to the MCTS-based generator. This suggests that the GA algorithm has the potential to find optimal states for game tutoring, provided that execution time issues are addressed through efficient computation techniques, such as multiprocessing.

APPENDIX B DEMONSTRATION AND CODEBASE

The demonstration and source code are available online.³ The repository includes the game environment used in the user study, along with implementations of the Random, MCTS-GEN, and MCTS-PSM agents. Crowdsourced gameplay logs were collected using the Google Firebase.⁴

³[Online]. Available: <https://github.com/bic4907/Personalized-Tutorial-Generation>

⁴[Online]. Available: <https://firebase.google.com/>

ACKNOWLEDGMENT

The authors would like to express their appreciation to PuzzleOne Studio and Jaeki Kim for their dedicated work on providing the game research platform.

REFERENCES

- [1] P. Sweetser and D. Johnson, "Evaluating the gameflow model with different stakeholders," in *Proc. Extended Abstr. Annu. Symp. Comput.-Hum. Interact. Play Companion Extended Abstr.*, 2019, pp. 697–703.
- [2] P. Sweetser Kyburz et al., "Gameflow 2020: 15 years of a model of player enjoyment," in *Proc. 32nd Australian Conf. Hum.-Comput.-Interact. (OzCHI'20)*, 2020, pp. 705–711.
- [3] P. Sweetser, D. Johnson, P. Wyeth, A. Anwar, Y. Meng, and A. Ozdowska, "Gameflow in different game genres and platforms," *Comput. Entertainment*, vol. 15, no. 3, pp. 1–24, 2017, doi: [10.1145/3034780](https://doi.org/10.1145/3034780).
- [4] P. Sweetser and D. Johnson, "Gameflow and player experience measures: An initial comparison of conceptual constructs," in *Proc. 31st Australian Conf. Hum.-Comput.-Interact.*, 2019, pp. 317–321.
- [5] R. Moreno and R. E. Mayer, "Cognitive principles of multimedia learning: The role of modality and contiguity," *J. Educ. Psychol.*, vol. 91, no. 2, pp. 358–368, 1999, doi: [10.1037/0022-0663.91.2.358](https://doi.org/10.1037/0022-0663.91.2.358).
- [6] E. Andersen et al., "The impact of tutorials on games of varying complexity," in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst.*, 2012, pp. 59–68, doi: [10.1145/2207676.2207687](https://doi.org/10.1145/2207676.2207687).
- [7] J. Frommel, K. Fahlbusch, J. Brich, and M. Weber, "The effects of context-sensitive tutorials in virtual reality games," in *Proc. Annu. Symp. Comput. Hum. Interact. Play*, New York, NY, USA: Association for Computing Machinery, 2017, pp. 367–375, doi: [10.1145/3116595.3116610](https://doi.org/10.1145/3116595.3116610).
- [8] A. Sarrafzadeh, S. Alexander, F. Dadgostar, C. Fan, and A. Bigdeli, "How do you know that I don't understand? A look at the future of intelligent tutoring systems," *Comput. Hum. Behav.*, vol. 24, no. 4, pp. 1342–1363, 2008.
- [9] Q. Zhang and C. J. MacLellan, "(A)I will teach you to play Gomoku: Exploring the use of game AI to teach people," in *Proc. 9th ACM Conf. Learn. Scale*, 2022, pp. 263–266.
- [10] M. C. Green, A. Khalifa, G. A. B. Barros, A. Nealen, and J. Togelius, "Generating levels that teach mechanics," in *Proc. 13th Int. Conf. Found. Digit. Games*, New York, NY, USA, 2018, pp. 1–8, doi: [10.1145/3235765.3235820](https://doi.org/10.1145/3235765.3235820).
- [11] M. C. Green, L. Mugrai, A. Khalifa, and J. Togelius, "Mario level generation from mechanics using scene stitching," in *Proc. IEEE Conf. Games*, 2020, pp. 49–56.
- [12] M. C. Green, A. Khalifa, Charity, and J. Togelius, "Persona-driven dominant/submissive map (PDSM) generation for tutorials," in *Proc. 17th Int. Conf. Found. Digit. Games*, 2022, pp. 1–10.
- [13] M. C. Green, A. Khalifa, G. A. B. Barros, T. Machado, A. Nealen, and J. Togelius, "AtDELFI: Automatically designing legible, full instructions for games," in *Proc. 13th Int. Conf. Found. Digit. Games*, New York, NY, USA, 2018, pp. 1–10, doi: [10.1145/3235765.3235790](https://doi.org/10.1145/3235765.3235790).
- [14] M. Allameh and L. Zaman, "Jessy: A conversational assistant for tutoring digital board games," in *Proc. Extended Abstr. 2021 Annu. Symp. Comput.-Hum. Interact. Play*, 2021, pp. 168–173, doi: [10.1145/3450337.3483496](https://doi.org/10.1145/3450337.3483496).
- [15] S. Earle, M. Edwards, A. Khalifa, P. Bontrager, and J. Togelius, "Learning controllable content generators," in *Proc. IEEE Conf. Games*, 2021, pp. 1–9.
- [16] Z. Jiang, S. Earle, M. Green, and J. Togelius, "Learning controllable 3D level generators," in *Proc. 17th Int. Conf. Found. Digit. Games*, pp. 1–9, New York, NY, USA, 2022, doi: [10.1145/3555858.3563273](https://doi.org/10.1145/3555858.3563273).
- [17] M. Graves et al., "Procedural content generation of angry birds levels using Monte Carlo tree search," Ph.D. dissertation, Elect. Comput. Eng., The Univ. Texas at Austin, Austin, TX, USA, 2016.
- [18] M. Stephenson and J. Renz, "Agent-based adaptive level generation for dynamic difficulty adjustment in Angry Birds," *arXiv:1902.02518*, 2019. [Online]. Available: <http://arxiv.org/abs/1902.02518>
- [19] B. De Kegel and M. Haahr, "Procedural puzzle generation: A survey," *IEEE Trans. Games*, vol. 12, no. 1, pp. 21–40, Mar. 2020, doi: [10.1109/TG.2019.2917792](https://doi.org/10.1109/TG.2019.2917792).
- [20] L. Poretski and A. Tang, "Press A to jump: Design strategies for video game learnability," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2022, pp. 1–26, doi: [10.1145/3491102.3517685](https://doi.org/10.1145/3491102.3517685).
- [21] M. Ibrahim, P. Sweetser, and A. Ozdowska, "Tutorial level design guidelines for 2D fighting games," in *Proc. 18th Int. Conf. Found. Digit. Games*, 2023, pp. 1–11.
- [22] D. Williams-King, J. Denzinger, J. Aycocock, and B. Stephenson, "The gold standard: Automatically generating puzzle game levels," in *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertainment* 2012, pp. 191–196, doi: [10.1609/aiide.v8i1.12529](https://doi.org/10.1609/aiide.v8i1.12529).
- [23] B. Kartal, N. Sohre, and S. Guy, "Generating Sokoban puzzle game levels with Monte Carlo tree search," in *Proc. IJCAI-16 Workshop Gen. Game Playing*, 2016, pp. 1–12.
- [24] B. Kartal, N. Sohre, and S. Guy, "Data driven Sokoban puzzle generation with Monte Carlo tree search," in *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertainment* Jun. 2021, pp. 58–64. [Online]. Available: <https://ojs.aaai.org/index.php/AIIDE/article/view/12859>
- [25] F. Rupp, M. Eberhardinger, and K. Eckert, "Simulation-driven balancing of competitive game levels with reinforcement learning," *IEEE Trans. Games*, vol. 16, no. 4, pp. 903–913, Dec. 2024.
- [26] K. Compton, A. Smith, and M. Mateas, "Anza island: Novel gameplay using ASP," in *Proc. 3rd workshop Procedural Content Gener. Games*, 2012, pp. 1–4, doi: [10.1145/2538528.2538539](https://doi.org/10.1145/2538528.2538539).
- [27] R. Van Der Linden, R. Lopes, and R. Bidarra, "Procedural generation of dungeons," *IEEE Trans. Comput. Intell. AI Games*, vol. 6, no. 1, pp. 78–89, Mar. 2014, doi: [10.1109/TCIAIG.2013.2290371](https://doi.org/10.1109/TCIAIG.2013.2290371).
- [28] J. Doran and I. Parberry, "A prototype quest generator based on a structural analysis of quests from four MMORPGs," in *Proc. 2nd Int. Workshop Procedural Content Gener. Games*, 2011, pp. 1–8, doi: [10.1145/2000919.2000920](https://doi.org/10.1145/2000919.2000920).
- [29] I. Padayachee, "Intelligent tutoring systems: Architecture and characteristics," in *Proc. 32nd Annu. SACLA Conf.*, 2002, pp. 1–8.
- [30] S. B. Gilbert, S. B. Blessing, and L. Blankenship, "The accidental tutor: Overlaying an intelligent tutor on an existing user interface," in *Proc. CHI2009 Extended Abstr. Hum. Factors Comput. Syst.*, 2009, pp. 4603–4608, doi: [10.1145/1520340.1520707](https://doi.org/10.1145/1520340.1520707).
- [31] L. Berdun, A. Amandi, and M. Campo, "An intelligent tutor for teaching software design patterns," *Comput. Appl. Eng. Educ.*, vol. 22, no. 4, pp. 583–592, 2014, doi: [10.1002/cae.20582](https://doi.org/10.1002/cae.20582).
- [32] B. Williford, M. Runyon, W. Li, J. Linsey, and T. Hammond, "Exploring the potential of an intelligent tutoring system for sketching fundamentals," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, New York, NY, USA, 2020, pp. 1–13, doi: [10.1145/3313831.3376517](https://doi.org/10.1145/3313831.3376517).
- [33] A. Bandura, W. H. Freeman, and R. Lightsey, "Self-efficacy: The exercise of control," *J. Cogn. Psychother.*, vol. 13, no. 2, pp. 158–166, 1999, doi: [10.1891/0889-8391.13.2.158](https://doi.org/10.1891/0889-8391.13.2.158).
- [34] C. B. Browne et al., "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012, doi: [10.1109/TCIAIG.2012.2186810](https://doi.org/10.1109/TCIAIG.2012.2186810).
- [35] J. Nielsen, *Usability Engineering*. San Francisco, CA, United States: Morgan Kaufmann, 1994.
- [36] Student, "The probable error of a mean," *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908.
- [37] A. Field, *Discovering Statistics Using IBM SPSS Statistics*. London, U.K.: SAGE, 2024.
- [38] R. F. DeVellis and C. T. Thorpe, *Scale Development: Theory and Applications*. Thousand Oaks, CA, USA: SAGE Publications, 2021.
- [39] J. W. Tukey, "Comparing individual means in the analysis of variance," *Biometrics*, vol. 5, no. 2, pp. 99–114, 1949.
- [40] I. Cohen et al., "Pearson correlation coefficient," *Noise Reduction Speech Process.*, vol. 5, pp. 1–4, 2009.



In-Chang Baek received the B.S. degree in computer science from Sejong University, Seoul, South Korea, in 2021. He is currently working toward the integrated M.S. and Ph.D. degrees in procedural content generation with the Department of AI Convergence, Gwangju Institute of Science and Technology, Gwangju, South Korea.

His research interests include artificial intelligence, reinforcement learning, procedural content generation, and general game artificial intelligence.



Taehwa Park received the B.S. degree in computer science from Sejong University, Seoul, South Korea in 2019, and the M.S. degree in cultural technology in 2021 from the Gwangju Institute of Science and Technology, where he is currently working toward the Ph.D. degree in procedural content generation with the Department of AI Convergence.

His research interests include procedural content generation and general game artificial intelligence.



Kyung-Joong Kim (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in computer science and artificial intelligence from Yonsei University, Seoul, South Korea, in 2000, 2002, and 2007, respectively.

He is a Full Professor with the Department of AI Convergence, Gwangju Institute of Science and Technology (GIST), Gwangju, South Korea. Prior to joining GIST, he worked as an Assistant/Associate Professor with the Department of Computer Science and Engineering, Sejong University, Seoul. Before that, he was a Postdoc Researcher with Computational

Synthesis Laboratory, Cornell University, Ithaca, NY, USA, working under the supervision of Prof. Hod Lipson.

Dr. Kim was a Co-General Chair of *IEEE Conference on Games 2020*.